

Enhancing Binary-State Network Reliability with Layer-Cut BAT-MCS

Wei-Chang Yeh 

Department of Industrial Engineering and Engineering Management, National Tsing Hua University

ARTICLE INFO

Keywords:

Binary-State Network Reliability, Monte Carlo Simulation (MCS)
Binary-Addition-Tree algorithm (BAT)
BAT-MCS
Supervector

ABSTRACT

This paper introduces layer-cut BAT-MCS, an enhanced algorithm for binary-state network reliability assessment. The original BAT-MCS integrates the deterministic Binary Addition Tree (BAT) algorithm with stochastic Monte Carlo simulation (MCS) in terms of the novel supervectors, creating a self-regulating mechanism that reduces variance and improves efficiency. Despite its advantages, BAT-MCS exhibits limitations in supervector selection methodology and computational complexity of approximate reliability calculations. The proposed layer-cut BAT-MCS addresses these weaknesses through a novel layer-cut approach for supervector selection that significantly outperforms traditional min-cut methods. This innovation simplifies MCS complexity while maintaining comprehensive network analysis capabilities. Extensive numerical experiments conducted on small and medium-sized binary-state networks demonstrate that layer-cut BAT-MCS achieves superior computational efficiency and accuracy compared to both traditional MCS and the original BAT-MCS implementations. The results indicate that the layer-cut technique provides a more efficient network decomposition strategy, substantially reducing both runtime and variance. These improvements make layer-cut BAT-MCS particularly valuable for reliability assessment of small-scale or sparse network systems where computational resources are limited and high accuracy is required.

1. Introduction

Binary-state networks represent the cornerstone of modern reliability engineering, where system components exist in one of two discrete states: operational or failed [1]. This elegant mathematical framework transforms complex system behaviors into analyzable discrete transitions, enabling precise reliability assessment across critical infrastructure domains [1,2]. As networks grow increasingly interconnected, binary-state modeling provides essential insights into system vulnerabilities and optimization opportunities that would otherwise remain hidden within system complexity.

The practical applications of binary-state reliability analysis have expanded dramatically—now encompassing wireless sensor networks [3], logistics network [4], transportation infrastructure [5], social networks connecting billions of users [6], air transport network [7], and advanced Markovian processes [8]. This proliferation of applications, coupled with exponential growth in network scale and structural complexity, has created unprecedented computational challenges that traditional methods struggle to address efficiently.

Network reliability—the probability of successful system functioning—serves as the fundamental metric driving system availability assessments [9,10], maintenance optimization [11,12], production

systems [13], and resource allocation decisions [14]. However, calculating exact reliability in binary-state networks represents an NP-hard problem [15–17], making Monte Carlo simulation (MCS) methods the de facto standard for practical reliability estimation [18–20]. Early researchers have explored the use of MCS in the context of network reliability [21,22] including sample stratification for variance reducing [23,24].

While conventional MCS approaches effectively address many reliability problems, they face significant limitations when confronting small-to-medium networks with heterogeneous components and complex interdependencies [25,26]. These limitations manifest in excessive computational requirements and high variance estimates—precisely where mission-critical applications demand maximum efficiency and accuracy.

Recent methodological advances have focused on hybrid techniques that leverage complementary strengths of deterministic and stochastic approaches [27]. The BAT-MCS method [25] represents a breakthrough in this direction by combining the structural precision of the binary-addition-tree algorithm (BAT) [28] with the flexible sampling capabilities of MCS [29].

BAT's hierarchical exploration of state space provides crucial structural guidance for simulation efforts, dramatically improving

E-mail address: yeh@iee.org.

<https://doi.org/10.1016/j.ress.2025.111446>

Received 20 April 2025; Received in revised form 7 July 2025; Accepted 9 July 2025

Available online 10 July 2025

0951-8320/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

efficiency compared to purely stochastic approaches [28]. Despite these advances, the original BAT-MCS method suffers from critical limitations: inflexible simulation parameters and suboptimal selection of super-vectors (essential state configurations), leaving significant performance potential untapped [25].

Our research introduces layer-cut BAT-MCS—a transformative approach that fundamentally reimagines super-vector selection through network topology analysis. This method introduces two critical innovations:

- 1. Stratified Layer-Cut Strategy:** Unlike conventional approaches that select super-vectors based on simplistic criteria, our method employs network-theoretic principles to identify critical configuration subspaces with maximum influence on reliability estimation.
- 2. Normalization-Based Approximation:** We develop a novel computational framework that intelligently redistributes simulation resources based on configuration importance, employing a normalization factor that dramatically enhances computational efficiency while preserving estimation accuracy.

By strategically prioritizing influential state vectors and deprioritizing marginal configurations, layer-cut BAT-MCS achieves unprecedented computational efficiency without sacrificing statistical rigor. This innovative approach renders previously intractable network reliability problems computationally feasible.

Extensive numerical experiments on diverse binary-state networks conclusively demonstrate that layer-cut BAT-MCS consistently outperforms both traditional MCS and the original BAT-MCS across critical performance metrics. Our approach exhibits particular strength in complex scenarios with heterogeneous reliability characteristics and intricate structural dependencies—precisely the conditions where conventional methods falter.

These substantial performance improvements establish layer-cut BAT-MCS as a robust, scalable solution for modern reliability engineering challenges, with applications spanning critical infrastructure protection, communication network optimization, and complex system design.

The remainder of this paper is structured as follows: [Section 2](#) establishes the formal nomenclature and foundational assumptions. [Section 3](#) presents a critical review of reliability estimation methodologies, focusing on MCS, BAT, PLSA, and BAT-MCS. [Section 4](#) introduces the layer-cut BAT-MCS methodology, comprehensively detailing its theoretical foundations and implementation. [Section 5](#) provides extensive experimental validation, performance benchmarking, and sensitivity analysis. [Section 6](#) concludes with implications for reliability engineering practice and outlines promising extensions to multi-state systems and dynamic reliability frameworks.

2. Acronyms, notations, nomenclature, and assumptions

This section outlines the essential acronyms, notations and definitions, and assumptions that form the foundation of the proposed layer-cut BAT-MCS methodology.

2.1. Acronyms

MCS	:	Monte Carlo simulation
BAT	:	Binary-Addition-Tree Algorithm
BAT-MCS	:	BAT-based MCS
layer-cut BAT-MCS	:	Proposed new cut-based BAT-MCS
LSA	:	Layered Search Algorithm
PLSA	:	Path-based LSA

2.2. Mathematical Notations and Definitions

2.2.1. Basic Mathematical Operations

$ \bullet $:	Cardinality (number of elements in a set)
$E[\bullet]$:	Expected value
Var	Variance
$[\bullet]$:	
$\Pr^*(\bullet)$:	Approximate probability
a_i :	the i th arc
V :	$V = \{1, 2, \dots, n\}$ is the set of nodes, where node 1 is the source node, and node n is the sink node
E :	$E = \{a_1, a_2, \dots, a_m\}$ is the set of arcs
$G(V, E)$:	a graph with V and E . For example, Figure 1 is a graph with $V = \{1, 2, 3, 4\}$,
E :	$E = \{a_1, a_2, a_3, a_4, a_5\}$, source node 1, and sink node 4

2.2.2. Network Components

$\Pr(a_i)$:	probability of arc a_i being in state 1 (working). For example, $\Pr(a_1) = 0.9$, $\Pr(a_2) = 0.8$, $\Pr(a_3) = 0.7$, $\Pr(a_4) = 0.6$, and $\Pr(a_5) = 0.5$ in Figure 1.
X :	state vector/supervector
$\underline{X}$:	the set or all supervectors where arc a is simultaneously present ($a \in \underline{X}$) and absent ($a \notin \underline{X}$) in supervector X .
$X(a_i)$:	state of arc a_i in vector X for $i = 1, 2, \dots, m$
$X(a_{[i]})$:	the i th arc selected in vector/supervector X
β :	number of arcs in the supervector for BAT-MCS
b :	number of arcs in the layer-cut for the proposed layer-cut BAT-MCS
$\mathbf{D}$:	state distribution $\mathbf{D}(a) = \Pr(a)$ for all $a \in E$, e.g., $\mathbf{D}$ is listed in Table 1.
$G(V, E, \mathbf{D})$:	a binary-state network can be modeled as a graph $G(V, E)$ with a state distribution $\mathbf{D}$. $G(V, E)$ with the $\mathbf{D}$. For example, Figure 1 illustrates such a network configuration where $\mathbf{D}$, as specified in Table 1, governs the system's operational states.
$G(X)$:	the subnetwork of G is defined as $G(X) = G(V, E(X), \mathbf{D})$, where $E(X) = \{a \in E \mid X(a) = 1\}$.
$0(X)$:	A zero subvector such the state of each arc in X is in a failed state (i.e., $x_i = 0$ for all $x_i \in X$).

Simulation Representations and Parameters

R_{MCS}	:	Reliability estimate derived from Monte Carlo simulation (MCS).
$R_{BAT-MCS}$	:	Reliability estimate obtained via the proposed BAT-MCS method.
R_{LCBM}	:	Reliability estimate obtained via the proposed layer-cut BAT-MCS method.
$R(G)$	:	Exact reliability of the network $G(V, E, \mathbf{D})$, where $\mathbf{D}$ denotes the state distribution.
$\Omega(C)$	:	$\Omega(C) = \{X \mid X \text{ is a supervector solving all arcs in set } C\}$. For example, if $C = \{a_1, a_3\}$, then $\Omega(C) = \{(x_1, x_3) \mid (0, 0), (1, 0), (0, 1), (1, 1)\}$.
$\Omega_1(C)$	:	$\Omega_1(C) = \left\{ X \mid X = (x_{[1]}, x_{[2]}, \dots, x_{[b]}) \in \Omega(C), x_{[k]} \in \{0, 1\}, \sum_{k=1}^b X(a_{[k]}) \geq 1 \right\}$
N_{sim}	:	Total number of simulations in a single run
$N_{sim}(X)$	:	Number of simulations for a specific supervector X in a single run
N_{pass}	:	Total number of connected vectors identified across all simulations in a single run
$N_{pass}(X)$	:	Number of connected vectors identified for a specific supervector X in a single run
N_{run}	:	Total number of total runs

Nomenclature

Binary-state network:	A network in which every arc (or link) can exist in one of two states, typically represented as 0 (failed) or 1 (operational).
Reliability:	The probability that a network operates successfully, meaning it provides a functioning path between specified nodes in a binary-state network.
Supervector:	A supervector is defined as a sub-vector $X = (x_1, x_2, \dots, x_\beta)$ such that for each index i (where $1 \leq \beta \leq m$), the relation $X(a_i) = x_i$ holds [36].

(continued on next page)

(continued)

Connect/disconnect vector:	A vector X is a connected/ disconnect if nodes 1 and n is connected in $G(X)$.
Subnetwork	$G(X)$ is a subnetwork induced by the binary vector X . This means $G(X)$ contains all vertices V from the original graph G , but only includes edges a for which the corresponding element in X equals 1, while maintaining the same state distribution $\mathbf{D}$. For example, in Figure 2, $G(X)$ represents the network configuration, where $X = (1, 1, 1, 1, 0)$. Figure 1 depicts the network structure $G(V, E)$, with $G(X)$ indicating the binary-state vector $(1, 1, 1, 1, 0)$, which defines the operational status of the network components. This configuration is derived from the base graph $G(V, E)$ illustrated in Figure 1, with X determining the active/ inactive states of edges or nodes.

2.4. Assumptions

1. Networks contain no parallel arcs or loops.
2. All nodes are assumed to be completely reliable and interconnected.
3. All arc states are statistically independent.

3. Review of BAT, PLSA, layer-cuts, MCS, and BAT-MCS

The proposed layer-cut BAT-MCS extends the BAT-MCS framework [25] by synthesizing four key methodologies: the BAT [28], PLSA [30], supervector [31], and MCS [17,32,33]. This section provides a systematic analysis of these foundational components and their integration, beginning with individual examinations of the theoretical foundations of BAT, core principles of PLSA, supervector, and statistical properties of MCS, followed by analysis of the unified BAT-MCS framework.

Special attention is given to the statistical advantages, specifically enhanced variance reduction and improved rare-event sampling capabilities, as well as the computational benefits of enhanced scalability and algorithmic efficiency achieved within the unified layer-cut BAT-MCS architecture.

3.1. BAT

The BAT, introduced by Yeh [28], provides a deterministic methodology for exhaustive state space enumeration in optimization problems, independent of solution feasibility. The algorithm initializes with a zero vector $X = (0, 0, \dots, 0)$ and generates subsequent state vectors through iterative application of two fundamental rules [28]:

1. **Saturation Rule:** Locate the first failed arc (denoted as a) in X . Reset the states of all arcs preceding a to zero and change the state of a to working to form a new state vector.
2. **Termination Rule:** If no failed arc exists, the algorithm terminates.

The BAT algorithm's elegance derives from its fundamental design principles of simplicity, computational efficiency, and adaptability [28]. Through systematic state vector transitions, it achieves comprehensive exploration of configuration spaces while optimizing computational resources. The algorithm maintains $O(2^{m-1})$ time complexity [34] and minimal memory overhead by eliminating redundant state storage [35]. The complete algorithmic implementation is detailed in the pseudocode below, with the full source code available in [28].

Algorithm: BAT	
Input:	m (the number of coordinates).
Output:	All possible binary-state m -tuple vectors.
STEP 0.	Initialize X as a zero m -tuple vector.
STEP 1.	Identify the first failed arc in X , say a_k . If no such arc exists, halts, and all m -tuple vectors have been generated.
STEP 2.	Let $X(a_k) = 1$, reset $x_j = 0$ for $j = 1, 2, \dots, k-1$, a new X is generated, and go to STEP 1.

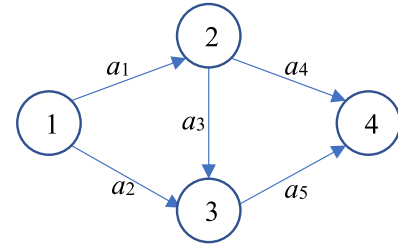


Fig 1. Example network.

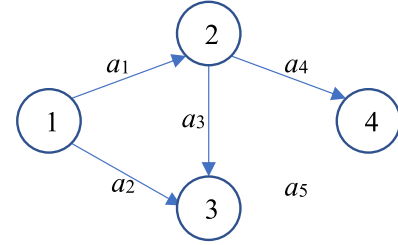


Fig 2. $G(X)$, where $X = (1, 1, 1, 1, 0)$ and $G(V, E)$ is shown in Figure 1.

Table 2 provides a detailed, step-by-step breakdown of the BAT procedure (as depicted in Figure 1), demonstrating its systematic exploration of binary-state vectors. The table includes columns $T(X)$, $C(X)$, and $W(X)$, which represent illustrative examples of time, cost, and weight, associated with state vector X . These functions are included solely for demonstration purposes and do not represent actual implementation functions.

3.2. Supervector and its Superfamily

The supervector $S_{\text{super}} = (a_1, a_2, \dots, a_\beta)$ represents a specialized subset of vectors, where each position i (ranging from 1 to $\beta \leq m$) denotes the operational state of the i -th arc [25,31]. Originally introduced in the quick BAT algorithm [31] and later incorporated into BAT-MCS [25], this construct optimizes processing efficiency by facilitating rapid identification and exclusion of disconnected vectors during component connectivity verification.

The associated superfamily $\Omega(S_{\text{super}}) = \{X \mid X = (x_1, x_2, \dots, x_\beta)$, where $x_i = 0, 1\}$ encompasses all possible solutions for the supervector $S_{\text{super}} = (a_1, a_2, \dots, a_\beta)$. This concept, first proposed in BAT-MCS [25], enables computational optimization by efficiently partitioning the solution space into feasible ($F(X) = 1$) and infeasible ($F(X) = 0$) configurations.

For example, in Figure 1, where $S_{\text{super}} = (a_1, a_2)$ as a cut and subvector. The corresponding configuration space $\Omega(S_{\text{super}})$ comprises all possible combinations of (x_1, x_2) , defined as $\Omega(a_1, a_2) = \{X \mid X = (x_1, x_2) = (0, 0), (1, 0), (0, 1), (1, 1)\}$, with cardinality $|\Omega(a_1, a_2)| = 2^2 = 4$.

3.3. MCS and Its Important Statistical Characteristics

Monte Carlo Simulation (MCS) is a widely adopted computational method for estimating solutions to high-dimensional, nonlinear, or computationally intractable problems [25,32]. For this study, the crude MCS approach—as detailed in [33,36]—was selected to approximate binary-state network reliability, chosen for its implementation simplicity and demonstrated effectiveness in reliability analysis [33,36]. The algorithm estimates system behavior through statistical sampling of network states, achieving an optimal balance between computational efficiency and estimation accuracy [32].

Consider a binary-state stochastic network $G(V, E, \mathbf{D})$, where V denotes the set of nodes, E represents the directed edges, and $\mathbf{D}$ defines all arc reliability parameters, with $\mathbf{D}(a) = \Pr(a)$ indicating the probability that arc a is operational. The network includes a source node $s = 1$ and a

Table 1
Probability distribution D.

i	$D(a_i) = \Pr(a_i)$
1	0.9
2	0.8
3	0.7
4	0.6
5	0.5

Table 2

All vectors obtained from the BAT on the network of Figure 1.

iteration	X	$T(X)$	$C(X)$	$W(X)$
1	(0, 0, 0, 0, 0)	19	19	27
2	(1, 0, 0, 0, 0)	12	13	29
3	(0, 1, 0, 0, 0)	11	18	10
4	(1, 1, 0, 0, 0)	14	21	24
5	(0, 0, 1, 0, 0)	17	11	14
6	(1, 0, 1, 0, 0)	13	10	16
7	(0, 1, 1, 0, 0)	10	11	24
8	(1, 1, 1, 0, 0)	18	28	38
9	(0, 0, 0, 1, 0)	17	23	24
10	(1, 0, 0, 1, 0)	16	16	10
11	(0, 1, 0, 1, 0)	14	22	26
12	(1, 1, 0, 1, 0)	19	22	25
13	(0, 0, 1, 1, 0)	18	12	28
14	(1, 0, 1, 1, 0)	11	11	25
15	(0, 1, 1, 1, 0)	14	24	22
16	(1, 1, 1, 1, 0)	12	11	15
17	(0, 0, 0, 0, 1)	14	26	28
18	(1, 0, 0, 0, 1)	17	16	36
19	(0, 1, 0, 0, 1)	15	10	28
20	(1, 1, 0, 0, 1)	11	26	15
21	(0, 0, 1, 0, 1)	16	13	38
22	(1, 0, 1, 0, 1)	19	16	31
23	(0, 1, 1, 0, 1)	10	17	20
24	(1, 1, 1, 0, 1)	18	29	27
25	(0, 0, 0, 1, 1)	16	24	19
26	(1, 0, 0, 1, 1)	14	13	28
27	(0, 1, 0, 1, 1)	17	28	34
28	(1, 1, 0, 1, 1)	12	29	28
29	(0, 0, 1, 1, 1)	12	25	35
30	(1, 0, 1, 1, 1)	19	22	16
31	(0, 1, 1, 1, 1)	11	18	21
32	(1, 1, 1, 1, 1)	15	28	31

sink node $t = n$. The nodes 1 and n are always the source and the destination nodes, respectively. Additionally, let N_{sim} denote the pre-defined number of MCS to be performed. The pseudocode for this crude

MCS implementation follows [37].

Algorithm: MCS

Input: $G(V, E, D)$ and N_{sim} .**Output:** Approximated binary-state network reliability R_{MCS} .**STEP 0.** Let $\text{sim} = i = 1$ and $N_{\text{pass}} = 0$.**STEP 1.** Let $X(a_i) = 1$ if $\rho \leq \Pr(a_i)$, else $X(a_i) = 0$, where ρ is a uniformly distributed random number in $[0, 1]$.**STEP 2.** If $i < m$, let $i = i + 1$ and go to STEP 1.**STEP 3.** If X is connected, let $N_{\text{pass}} = N_{\text{pass}} + 1$.**STEP 4.** If $\text{sim} < N_{\text{sim}}$, let $\text{sim} = \text{sim} + 1$, $i = 1$, and go to STEP 1.**STEP 5.** Let $R_{\text{MCS}} = N_{\text{pass}}/N_{\text{sim}}$ and halt.The MCS estimator R_{MCS} is unbiased and consistent [25,33], with

$$E[R_{\text{MCS}}] = R \quad (1)$$

$$\text{Var}[R_{\text{MCS}}] = \frac{R(1-R)}{N_{\text{sim}}}, \quad (2)$$

where R is the exact binary-state network reliability. To achieve a desired tolerable error bound ε (half-width of the confidence interval) at a $(1-\alpha)\%$ confidence level, the required number of simulations N_{sim} satisfies [32,33,37]:

$$R(1-R) \left[\frac{\Phi^{-1}\left(1 - \frac{\alpha}{2}\right)}{\varepsilon} \right]^2 \leq N_{\text{sim}}, \quad (3)$$

where $\Phi^{-1}\left(1 - \frac{\alpha}{2}\right)$ is the quantile of the standard normal distribution (Z-score). These properties are rigorously proven in foundational MCS literature [33,36].

To illustrate the methodology, consider the network structure in Figure 1, where arc state probabilities are provided in Table 1. A MCS framework with $N_{\text{sim}} = 16$ trials was applied, and the results are summarized in Table 3. For each simulation, the value of x_i (representing the state of arc i) is dynamically assigned based on the corresponding probability ρ_i , where $i = 1, 2, 3, 4, 5$, as outlined in STEP 1 of the MCS pseudocode.

Table 3 reveals a critical limitation arising from redundant sampling instances (e.g., $X_4 = X_{14}$, $X_8 = X_{13}$, $X_{11} = X_{16}$, $X_{12} = X_{15}$, $X_2 = X_5$, $X_7 = X_9$, where X_i is the vector obtained in simulation trials i), which occur due to an insufficiently small N_{sim} . This inadequate sampling diversity results in the overrepresentation of specific system states, thereby compromising the accuracy of reliability estimates. These findings emphasize the imperative of selecting N_{sim} in accordance with Eq. (3) to ensure statistical robustness.

The analysis reveals that $N_{\text{pass}} = 13$ because there are 13 simulated state vectors connected across all permutations of X . Consequently, $R_{\text{MCS}} = N_{\text{pass}}/N_{\text{sim}} = 12/16 = 0.750$ (see Table 3). This value diverges from

Table 3

MCS Procedure for Figure 1 based on Table 1.

i	ρ_1	ρ_2	ρ_3	ρ_4	ρ_5	x_1	x_2	x_3	x_4	x_5	Connect?
1	0.92168	0.71974	0.73371	0.40070	0.15396	0	1	0	1	1	No
2	0.40533	0.33365	0.66730	0.83186	0.01404	1	1	1	0	1	Yes
3	0.71173	0.83719	0.66681	0.31019	0.47161	1	0	1	1	1	Yes
4	0.33772	0.36953	0.55257	0.62505	0.83836	1	1	1	0	0	No
5	0.39771	0.73576	0.35345	0.86882	0.08958	1	1	1	0	1	Yes
6	0.78790	0.43831	0.72763	0.14010	0.01628	1	1	0	1	1	Yes
7	0.56750	0.02068	0.04221	0.40355	0.28200	1	1	1	1	1	Yes
8	0.78631	0.79431	0.96743	0.57409	0.86514	1	1	0	1	0	Yes
9	0.34288	0.56388	0.52413	0.17044	0.41365	1	1	1	1	1	Yes
10	0.94191	0.66559	0.92689	0.86024	0.49946	0	1	0	0	1	No
11	0.19682	0.65801	0.28775	0.15021	0.93827	1	1	1	1	0	Yes
12	0.31759	0.88889	0.38306	0.99749	0.31031	1	0	1	0	1	Yes
13	0.75611	0.33225	0.90883	0.33284	0.89182	1	1	0	1	0	Yes
14	0.02275	0.35458	0.20985	0.98929	0.54591	1	1	1	0	0	No
15	0.20804	0.99488	0.07421	0.93896	0.18774	1	0	1	0	1	Yes
16	0.75760	0.00952	0.14548	0.54478	0.78840	1	1	1	1	0	Yes

Table 4

Finding all layers (via PLSA) and layer-cuts.

i	L_i	c_i
1	{1}	$\{a_1, a_2\}$
2	{2, 3}	$\{a_4, a_5\}$
3	{4}	

Table 5 $N_{sim}(X)$ and $Pr(X)$ for Figure 1 based on Table 1.

$X = (x_1, x_2)$	(0, 0)	(1, 0)	(0, 1)	(1, 1)
$Pr(X)$	$0.1 \times 0.2 = 0.02$	$0.9 \times 0.2 = 0.18$	$0.1 \times 0.8 = 0.08$	$0.9 \times 0.8 = 0.72$
$Pr(X) / \sum_X Pr(X)$	0.02	0.18	0.08	0.72
$N_{sim} \times Pr(X) / \sum_X Pr(X)$	0.32	2.88	1.28	11.52
$N_{sim}(X)$	1	3	2	10

the exact system reliability of 0.72461, a discrepancy attributable to the limited simulation size ($N_{sim} = 16$) and the adoption of a simplistic MCS framework that fails to incorporate solution quality metrics.

3.4. PLSA in verifying the connectness and finding layers

The Path-Based Layered Search Algorithm (PLSA) evolved from the Layered Search Algorithm (LSA) [31], which was originally designed for identifying d -minimal paths in acyclic networks [31]. While LSA's core framework offers computational efficiency and simplicity, it has spawned several specialized variants: PLSA for source-to-sink connectivity verification [28], Group Layered Search Algorithm (GLSA) for multi-node connectivity [1], and Total Layered Search Algorithm (TLSA) for full-network connectivity validation [38].

The PLSA, developed by the author in Reference [28], is a well-established path-based layered search method for network optimization (e.g., identifying d -minimal paths in acyclic networks) and adapts LSA's layered-search mechanism with a fundamental principle: any node n added to layers is necessarily connected to the source node (node 1). This approach eliminates redundant path evaluations, enhancing computational efficiency particularly for small-to-medium network reliability analyses [28]. The detailed implementation is presented in the pseudocode below [28].

Algorithm: PLSA	
Input:	A binary-state vector X , defining arc states in network $G(X)$.
Output:	Determines whether nodes 1 and n are connected in $G(X)$.
STEP 0.	Initialize the first layer $L_1 = \{1\}$, $i = 2$, and $L_2 = \emptyset$.
STEP 1.	Let layer $L_i = \{v \in V \mid \text{if } X(e_{u,v}) > 0, u \in L_{i-1}, \text{ and } v \notin \cup_{k < i} L_k \text{ in } G(X)\}$.
STEP 2.	If $n \in L_i$, terminate and $G(X)$ is connected.
STEP 3.	If $L_i = \emptyset$, terminate and $G(X)$ is disconnected. Otherwise, let $i = i + 1$, $L_i = \emptyset$, and go to STEP 1.

The PLSA implements a breadth-first search (BFS)-inspired traversal that partitions nodes into discrete layers while ensuring single-visit node exploration [30]. The algorithm achieves $O(n)$ time complexity, where n represents the number of nodes, due to its guaranteed termination within $n-1$ iterations—the maximum possible path length in an n -node network. This linear complexity ensures efficient scalability for small-to-medium networks through systematic layer-wise expansion from the source node [30].

To demonstrate, consider the capacity vector $X = (2, 4, 4, 1, 4)$ from Figure 1. The PLSA's layer-wise expansion results, documented in the " L_i " column of Table 4, confirm network connectivity.

3.5. BAT-MCS

In the traditional BAT-MCS framework [25], the system reliability

$R_{BAT-MCS}$ is approximated using the following equation:

$$R_{BAT-MCS} = \sum_{\forall X} Pr(X) \times Pr^*(\underline{X}), \quad (4)$$

where

- each supervector $X = (x_1, x_2, \dots, x_\beta)$ is obtained from the BAT and its probability is defined:

$$Pr(X) = \sum_{\forall x_k=1} Pr(a_k) \times \sum_{\forall x_k=0} [1 - Pr(a_k)], \quad (5)$$

- $Pr_{MCS}(\underline{X})$ is the approximated probability obtained from the MCS after known X :

$$Pr_{MCS}(X) = \frac{N_{pass}(X)}{N_{sim}(X)}, \quad (6)$$

- the total number of simulations allocated to X :

$$N_{sim}(X) = N_{sim} \frac{Pr(X)}{\sum_{\forall X} Pr(X)}. \quad (7)$$

Eq. (4) aggregates the contributions of all vectors X , weighted by their respective probabilities and simulation-based failure rates. Computing $Pr(X)$ using Eq. (5) requires b multiplications to obtain $Pr(X)$ for each possible X . For $Pr_{MCS}(X)$ in Eq. (6), each calculation requires one division operation. Calculating $N_{sim}(X)$ involves one multiplication operation, followed by another multiplication after obtaining $\sum_{\forall X} Pr(X)$. Therefore, the total computational requirements for $R_{BAT-MCS}$ in Eq. (4) sum to:

- Multiplications: $(\beta \times 2^\beta + 2 \times 2^\beta + 1)$ operations
- Divisions: $2^\beta + 1$ operations
- Summations: $2^\beta + 1$ operations

Let R_S represent the conditional reliability given state S , and we're using the binomial variance formula. A key limitation of this approach is the variance of $R_{BAT-MCS}$ [25]:

$$Var[R_{BAT-MCS}] = \frac{R_S(1 - R_S)}{N_{sim}} \sum_{S \in F_C} Pr^2(S) < Var[R_{MCS}] \quad (8)$$

cannot be analytically derived from the equation, where F_C is the set of frontier (or uncertain) states that are not classified as definitely reliable or unreliable, so Monte Carlo simulation is applied to estimate their reliability contribution. This necessitates the use of empirical methods for error estimation.

To elucidate the fundamental distinctions between conventional MCS and the BAT-MCS, we revisit the network in Figure 1 under identical experimental conditions:

- The simulation size is fixed at $N_{sim} = 16$ trials for direct methodological comparison.
- Arc state probabilities $Pr(a_i)$ for $i=1, 2, 3, 4, 5$ are retained from Table 1
- Uniformly distributed random number $\rho_i \in [0, 1]$ for $i=1, 2, 3, 4, 5$ are from Table 3.

In conventional MCS, each trial independently samples the state x_i of all arcs ($i = 1, 2, 3, 4, 5$) using their respective ρ_i . In contrast, BAT-MCS strategically partitions the problem into supervector and its family, e.g., supervector $S_{super} = (a_1, a_2)$ and superfamily $\Omega(S_{super}) = \{(x_1, x_2) \mid (0, 0),$

Table 6BAT-MCS for Figure 1 based on Table 3 and Table 5, where $X = (x_1, x_2, x_3, x_4, x_5)$.

(x_1, x_2)	i	p_3	p_4	p_5	x_3	x_4	x_5	Connect?	$N_{\text{pass}}(X)/N_{\text{sim}}(X)$
(0, 0)	1	0.73371	0.40070	0.15396	0	1	1	No	0/1
(1, 0)	1	0.66730	0.83186	0.01404	1	0	1	Yes	2/3
	2	0.66681	0.31019	0.47161	1	1	1	Yes	
	3	0.55257	0.62505	0.83836	1	0	0	No	
(0, 1)	1	0.35345	0.86882	0.08958	1	0	1	Yes	1/2
	2	0.72763	0.14010	0.01628	0	1	1	No	
(1, 1)	1	0.04221	0.40355	0.28200	1	1	1	Yes	8/10
	2	0.96743	0.57409	0.86514	0	1	0	Yes	
	3	0.52413	0.17044	0.41365	1	1	1	Yes	
	4	0.92689	0.86024	0.49946	0	0	1	No	
	5	0.28775	0.15021	0.93827	1	1	0	Yes	
	6	0.38306	0.99749	0.31031	1	0	1	Yes	
	7	0.90883	0.33284	0.89182	0	1	0	Yes	
	8	0.20985	0.98929	0.54591	1	0	0	No	
	9	0.07421	0.93896	0.18774	1	0	1	Yes	
	10	0.14548	0.54478	0.78840	1	1	0	Yes	

(1, 0), (0, 1), (1, 1)).

As demonstrated in Table 5, BAT-MCS evaluates these three super-vectors $(x_1, x_2) = (0, 0)$, $(1, 0)$, $(0, 1)$, and $(1, 1)$ alongside their joint probabilities $\Pr(X)$ and allocated simulation counts $N_{\text{sim}}(X)$.

Applying Eq. (4) in BAT-MCS based on the results from Table 6 yields:

$$R_{\text{BAT-MCS}} = 0.02 \times 0/1 + 0.18 \times 2/3 + 0.08 \times 1/2 + 0.72 \times 8/10 = 0.7360. \quad (9)$$

Despite the limited simulation size ($N_{\text{sim}} = 16$), the $R_{\text{BAT-MCS}}$ result (0.7360) demonstrates closer alignment with the exact system reliability (0.72461) compared to the conventional MCS estimate ($R_{\text{MCS}} = 0.750$, Table 3). This improvement arises from BAT-MCS's variance-reduction strategy, which strategically prioritizes high-probability supervectors (e.g., $(x_1, x_2) = (1, 1) = 0.72$) while excluding deterministic failures (e.g., network cuts like $(x_1, x_2) = (0, 0)$).

4. The proposed layer-cut BAT-MCS

The layer-cut BAT-MCS framework is introduced as a computational engine to integrate deterministic and stochastic methods. The deterministic component leverages layer-cut decomposition derived from PLSA and its associated family of cuts generated via BAT. The stochastic component employs an MCS scheme, augmented with a novel approximation formula to compute system reliability efficiently. This hybrid methodology synthesizes structural insights from PLSA/BAT with probabilistic sampling via MCS, ensuring both computational tractability and statistical fidelity. The interplay of these components, along with their theoretical and algorithmic implications, is rigorously discussed in this section.

4.1. Layer-Cut

For an undirected graph $G(V, E)$, a cut is a partition of the node set V into two disjoint node subsets S and T such that $S \cup T = V$ and $S \cap T = \emptyset$. A layer-cut is a specialized cut derived from the layered structure generated by the PLSA [28]. Specifically, it consists of arcs connecting nodes in two consecutive layers L_i and L_{i+1} obtained during PLSA's traversal. This concept, first proposed in [28], leverages the PLSA's iterative layer expansion to systematically identify critical edge sets that partition the graph. The pseudocode in finding all layer-cuts is listed below:

Algorithm:	Finding All Layer-Cuts
Input:	A graph $G(V, E)$, the source node 1, and the sink node n .
Output:	All layer-cuts.
STEP 0.	Find all layers $L_1, L_2, \dots, L_\lambda$ using PLSA.
STEP 1.	Find all layer-cuts $c_k = \{e_{ij} \mid \text{nodes } i \text{ and } j \text{ are in } L_k \text{ and } L_{k+1}, \text{ seperately}\}$ for $k = 1, 2, \dots, (\lambda-1)$.
STEP 2.	Let $c_\lambda = E - \bigcup_{k=1}^{\lambda-1} c_k$.

The computational complexity for identifying all layer-cuts is $O(n+m)$, where n is the number of nodes and m is the number of arcs. Specifically, STEP 0 implements the PLSA with $O(n)$ complexity, as detailed in Section 3.2, while STEPs 1 and 2 require $O(m)$ operations as they traverse each arc exactly once.

From the analysis above, PLSA's layered structure demonstrates superior computational efficiency compared to traditional min-cut approaches, which require $O(mn^2)$ complexity when implemented using Karger's algorithm. Additionally, PLSA's constraint requiring at least one active arc per layer-cut significantly reduces the solution space from 2^b possible combinations (where b represents the total arc count) to a substantially smaller feasible set, thereby improving both computational tractability and solution quality.

To illustrate this process, consider the network depicted in Figure 1. The PLSA algorithm identifies three distinct layers: $L_1 = \{1\}$, $L_2 = \{2, 3\}$, and $L_3 = \{4\}$, as shown in the " L_i " column of Table 4. This layering structure yields two layer-cuts: $c_1 = \{a_1, a_2\}$ and $c_2 = \{a_4, a_5\}$, presented in Table 4's rightmost column. Both c_1 and c_2 represent valid cuts in Figure 1, as they effectively partition the graph into disjoint node subsets.

4.2. Cut Superfamily

The theoretical bridge between deterministic and stochastic approaches in the proposed layer-cut BAT-MCS is established through cut-based superfamilies. On the contrast, the conventional BAT-MCS framework employs an oversimplified selection criterion for super-vectors, which may compromise computational efficiency and solution accuracy. To address this limitation, we propose layer-cut BAT-MCS, a novel method grounded in the layer-cut superfamily—a concept introduced herein to systematically identify and discard infeasible superfamilies, thereby enhancing algorithmic performance.

The layer-cut superfamily synthesizes three methodologies: PLSA (Section 3.2), layer-cut concept (Section 5.1), and superfamily generation (Section 3.3). Formally, for a given layer-cut $C_{\text{super}} = (a_{[1]}, a_{[2]}, \dots, a_{[b]})$ —a set of arcs whose removal disconnects source node 1 and terminal node n —the associated layer-cut superfamily $\Omega_1(C_{\text{super}})$ is defined as:

$$\Omega_1(C_{\text{super}}) = \left\{ X \mid X = (x_{[1]}, x_{[2]}, \dots, x_{[b]}), x_{[k]} \in \{0, 1\}, \sum_{k=1}^b X(a_{[k]}) \geq 1 \right\}. \quad (10)$$

where $X(a_{[k]}) = x_{[k]}$ denotes the state of arc the k th arc $a_{[k]}$ in X . By

construction, $\Omega_1(C_{\text{super}})$ enforces at least one active arc per layer-cut, reducing the solution space cardinality from 2^b (where b is the total arc count) to a smaller set.

Example 1. Consider layer-cuts (a_1, a_2) and (a_4, a_5) from Section 5.1. The cut superfamily for both cases is $\Omega_1(a_1, a_2) = \Omega_1(a_4, a_5) = \{(0, 1), (1, 0), (1, 1)\}$, which contains only 3 elements compared to the $2^2 = 4$ possible combinations. This demonstrates the inherent reduction capability of the cut superfamily, pruning infeasible configurations a priori.

This methodological advancement ensures rigorous feasibility guarantees while streamlining computational complexity—a critical advantage for small-to-medium network analyses.

4.3. The Normalization Factor

While layer-cut BAT-MCS retains the core reliability approximation framework of BAT-MCS (Eq. (4)), it introduces critical modifications to the vector definition and sampling strategy to enhance computational efficiency.

In BAT-MCS, the system state X is represented as a subvector $X = (x_1, x_2, \dots, x_\beta)$, where β denotes the number of arcs in a single supervector. By contrast, layer-cut BAT-MCS redefines X as a full-dimensional layer-cut $X = (x_{[1]}, x_{[2]}, \dots, x_{[b]})$, where b corresponds to the total number of arcs in the layer-cut topology, constrained by the layer-cut superfamily $\Omega_1(C_{\text{super}})$. This ensures $X \in \Omega_1(C_{\text{super}})$, where $C_{\text{super}} = (a_{[1]}, a_{[2]}, \dots, a_{[b]})$ represents a layer-cut derived from PLSA, and guarantees the exclusion of trivial/infeasible states (e.g., the zero vector $(0, 0, \dots, 0)$), as

$$\sum_{k=1}^b x_{[k]} \geq 1, \quad (11)$$

is enforced by construction.

To optimize computational efficiency and quantify the statistical variance of BAT-MCS, we propose a novel mathematical formulation under the assumption that

$$\gamma = \frac{\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X)}{N_{\text{sim}}} = \frac{1 - \Pr(0(C_{\text{super}}))}{N_{\text{sim}}} \quad (12)$$

yields an integer value. To optimize computational efficiency and quantify the statistical variance of the results, we propose a novel mathematical formulation:

$$\begin{aligned} R_{\text{LCBM}} &= \sum_{X \in \Omega_1(C_{\text{super}})} \left[\Pr(X) \times \Pr_{\text{MCS}}(\underline{X}) \right] \\ &= \sum_{X \in \Omega_1(C_{\text{super}})} \left[\Pr(X) \times \frac{N_{\text{pass}}(X)}{N_{\text{sim}}(X)} \right] \\ &= \sum_{X \in \Omega_1(C_{\text{super}})} \left[\Pr(X) \times \frac{N_{\text{pass}}(X)}{\Pr(X)} \right] \\ &= \sum_{X \in \Omega_1(C_{\text{super}})} \left[\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X) \times \frac{N_{\text{pass}}(X)}{N_{\text{sim}}} \right] \\ &= \sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X) \times \sum_{X \in \Omega_1(C_{\text{super}})} \frac{N_{\text{pass}}(X)}{N_{\text{sim}}} \\ &= \gamma \left[\sum_{X \in \Omega_1(C_{\text{super}})} N_{\text{pass}}(X) \right] \\ &= \gamma N_{\text{pass}}. \end{aligned} \quad (13)$$

The computational efficiency is achieved through a single

calculation of $\frac{\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X)}{N_{\text{sim}}}$ requiring $|\Omega_1(C_{\text{super}})| = 2^b - 1$ summations and one division operation. The total computational cost comprises one multiplication, one division, and $2|\Omega_1(C_{\text{super}})|$ summations. This demonstrates that calculating R_{LCBM} is computationally more efficient than computing $R_{\text{BAT-MCS}}$, which takes $(\beta \times 2^\beta + 2 \times 2^\beta + 1)$ multiplications operations, $2^\beta + 1$ divisions operations, and $2^\beta + 1$ summations operations.

Furthermore, Eq. (12) facilitates the direct calculation of variance for the reliability approximation obtained through layer-cut BAT-MCS:

$$\begin{aligned} \text{Var}[R_{\text{LCBM}}] &= \text{Var} \left[\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X) \times \sum_{X \in \Omega_1(C_{\text{super}})} \frac{N_{\text{pass}}(X)}{N_{\text{sim}}} \right] \\ &= \left[\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X) \right]^2 \text{Var} \left[\sum_{X \in \Omega_1(C_{\text{super}})} \frac{N_{\text{pass}}(X)}{N_{\text{sim}}} \right] \\ &= \left[\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X) \right]^2 \text{Var}[R_{\text{BAT-MCS}}] \end{aligned} \quad (14)$$

where $\Pr(X)$ is the reliability of state X . This formulation leverages the reduced cardinality of $\Omega_1(C_{\text{super}})$ (achieved by discarding infeasible states) to minimize redundant computations while preserving statistical rigor.

4.4. Determine Maximal-Probability Minimal-Size C_{super} based on the Eq. (13)

In the proposed layer-cut BAT-MCS framework, the selection of the critical superset cut C_{super} from available layer-cuts is pivotal to enhancing computational efficiency. Let $C_{\text{super}} = (a_{[1]}, a_{[2]}, \dots, a_{[b]})$ represent a layer-cut identified via PLSA. As derived from Eq. (13), the probability $\Pr(0(C_{\text{super}})) = 1 - \sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X) = \prod_{k=1}^b [1 - \Pr(a_{[k]})]$ governs the variance reduction mechanism: an increase in $\Pr(0(C_{\text{super}}))$ directly reduces both $\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X)$ and the variance $\text{Var}[R_{\text{LCBM}}]$.

The failure state set $\Omega_1(C_{\text{super}})$ has a cardinality bounded by 2^{b-1} , where b denotes the number of arcs in C_{super} . This bound establishes a direct exponential relationship between b and the size of $\Omega_1(C_{\text{super}})$: as b increases, the number of X in $\Omega_1(C_{\text{super}})$ grows exponentially. While this introduces computational challenges, the expanded set enables a more comprehensive exploration of X . Specifically, for a fixed total number of MCS trials N_{sim} , the larger $\Omega_1(C_{\text{super}})$ allows finer-grained sampling across $\underline{X}$, improving the statistical resolution of the reliability estimate R_{LCBM} . This enhanced coverage reduces variance in the estimator, particularly when N_{sim} scales proportionally to $|\Omega_1(C_{\text{super}})|$, thereby balancing computational effort with accuracy.

From the above, the optimal layer-cut selection follows a two-tier hierarchical criteria system:

- **Primary Criterion:** Minimize the number of arcs in C_{super} , i.e., $|C_{\text{super}}|$.
- **Secondary Criterion:** Among layer-cuts with equal number of arcs, select the one with maximal $\Pr(0(C_{\text{super}}))$, where each coordinate in $\Pr(0(C_{\text{super}}))$ represents the probability of inactive states.

Algorithm: Finding the Maximal-Probability Minimal-Size Layer-Cut

Input: A graph $G(V, E)$, all layer-cuts $c_1, c_2, \dots, c_{k-1}$, the source node 1, and the sink node n .

Output: The maximal layer-cut C_{super} .

STEP 0. Find c^* such that $||c|| \leq ||c^*||$ for all $c \in \{c_1, c_2, \dots, c_{k-1}\}$, where $||c||$ is the size metric (arcs) of a layer-cut $||c||$.

STEP 1. If there is only one c^* , let $C_{\text{super}} = c^*$ and halt.

STEP 2. Find the layer-cut $C_{\text{super}} = c_b$ with the maximal probability among these c^* such that $||c_b|| = ||c^*||$ for all c_b and c^* .

Table 7
layer-cut BAT-MCS for Figure 1.

(x_4, x_5)	i	ρ_1	ρ_2	ρ_3	x_1	x_2	x_3	Connect?	$N_{\text{pass}}(X)$
(1, 0)	1	0.92168	0.71974	0.73371	0	1	0	No	5
	2	0.40533	0.33365	0.66730	1	1	1	Yes	
	3	0.71173	0.83719	0.66681	1	0	1	Yes	
	4	0.33772	0.36953	0.55257	1	1	1	Yes	
	5	0.39771	0.73576	0.35345	1	1	1	Yes	
	6	0.78790	0.43831	0.72763	1	1	0	Yes	
(0, 1)	1	0.56750	0.02068	0.04221	1	1	1	Yes	2
	2	0.78631	0.79431	0.96743	1	1	0	No	
	3	0.34288	0.56388	0.52413	1	1	1	Yes	
	4	0.94191	0.66559	0.92689	0	1	0	No	
(1, 1)	1	0.19682	0.65801	0.28775	1	1	1	Yes	6
	2	0.31759	0.88889	0.38306	1	0	1	Yes	
	3	0.75611	0.33225	0.90883	1	1	0	Yes	
	4	0.02275	0.35458	0.20985	1	1	1	Yes	
	5	0.20804	0.99488	0.07421	1	0	1	Yes	
	6	0.75760	0.00952	0.14548	1	1	1	Yes	

This hierarchical approach optimizes both variance reduction and computational efficiency. To illustrate this selection process, we examine two candidate layer-cuts from Section 3.3: $c_1 = \{a_1, a_2\}$ and $c_2 = \{a_4, a_5\}$. Computing the joint inactive probabilities using the binary-state probabilities presented in Table 1:

- For c_1 : $\Pr(x_1=0, x_2=0)=(1-0.9) \times (1-0.8)=0.1 \times 0.2=0.02$.
- For c_2 : $\Pr(x_4=0, x_5=0)=(1-0.6) \times (1-0.5)=0.4 \times 0.5=0.20$.

While c_2 exhibits a higher joint probability (0.20), indicating a potentially more effective variance reduction target, c_2 is ultimately selected as C_{super} . This selection reflects the hierarchical framework's emphasis on balancing computational tractability with probability-based optimization criteria.

4.5. Pseudocode for layer-cut BAT-MCS Reliability Approximation

The pseudocode for computing the approximate reliability R_{LCBM} of a binary-state network using the layer-cut BAT-MCS framework is presented below. This algorithm integrates Eqs. (12) and (13), combining deterministic layer-cut superfamily analysis (Section 4.4) with stochastic Monte Carlo sampling (Section 4.3) to balance computational efficiency and statistical rigor.

Algorithm: layer-cut BAT-MCS
Input: A binary-state stochastic network $G(V, E, D)$, where:
 V : Set of nodes, with source node $s = 1$ and sink node $t = n$.
 E : Set of directed arcs.
 $D(a) = \Pr(a)$ for all a denotes the operational probability of arc a for all $a \in E$.
 N_{sim} : Predefined number of Monte Carlo simulations.
Output: Approximate reliability R_{LCBM} .
STEP 0. Identification of C_{super} and calculate γ :

- Identify the maximal-probability minimal-size layer-cut $C_{\text{super}} = (a_{[1]}, a_{[2]}, \dots, a_{[b]})$ using the algorithm in Section 4.4.
- Determine the superfamily $\Omega_1(C_{\text{super}})$ defined in Eq. (10) using BAT.
- For each $X \in \Omega_1(C_{\text{super}})$, calculate the probability of supervector $\Pr(X)$.
- Calculate the normalization factor $\gamma = \frac{\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X)}{N_{\text{sim}}}$.

STEP 1. Aggregate results: $N_{\text{pass}} = \sum_{X \in \Omega_1(C_{\text{super}})} N_{\text{pass}}(X)$ using the MCS discussed in Section 3.4.
STEP 2. Compute the approximate reliability $R_{\text{LCBM}} = \gamma N_{\text{pass}}$ and halt.

The layer-cut BAT-MCS algorithm exhibits a total time complexity of $O(nN_{\text{sim}}(m-b)2^{b-1})$, dominated by STEP 1. In STEP 0, identifying the maximal-probability minimal-size layer-cut C_{super} (size b), computing its superfamily $\Omega_1(C_{\text{super}})$, (cardinality 2^{b-1}), and calculating the normalization factor γ contribute $O(b2^{b-1})$, primarily due to the enumeration and probabilistic evaluation of 2^{b-1} supervectors in $\Omega_1(C_{\text{super}})$.

STEP 1, however, incurs $O(nN_{\text{sim}}(m-b)2^{b-1})$ operations, as it involves N_{sim} MCS trials for each supervector $X \in \Omega_1(C_{\text{super}})$, requiring $(m-b)$

random variable generations and $O(n)$ -time connectivity verification via the PLSA per trial. STEP 2, which computes the final reliability R_{LCBM} through a scalar multiplication, adds negligible $O(1)$ overhead.

The exponential dependence on b and linear scaling with n , m , and N_{sim} highlight the method's tractability for systems with the maximal-probability minimal-size C_{super} and its suitability for small-to-medium networks when b is bounded. The size of C_{super} is a critical factor in the algorithm's time complexity. The selection of $b = |C_{\text{super}}|$ significantly impacts both computational efficiency and estimator variance.

While the proposed layer-cut BAT-MCS and BAT-MCS share similar asymptotic time complexity, layer-cut BAT-MCS demonstrates superior operational efficiency. Specifically, layer-cut BAT-MCS reduces the number of multiplicative and summative operations by 2^b compared to BAT-MCS, as detailed in Section 4.3. Fewer operations minimize cumulative numerical errors, enhancing precision. In terms of variance, layer-cut BAT-MCS is more robust due to the inequality:

$$\left[\sum_{X \in \Omega_1(C_{\text{super}})} \Pr(X) \right]^2 \leq \left[\sum_{X \in \Omega_1(S_{\text{super}})} \Pr(X) \right]^2. \quad (15)$$

which reflects tighter probability bounds on the critical failure modes.

4.6. Layer-Cut BAT-MCS in Step-By-Step Procedure

To systematically investigate the fundamental differences among conventional MCS, BAT-MCS, and the proposed layer-cut BAT-MCS approaches, we conducted a comparative analysis using the network depicted in Figure 1 under controlled experimental conditions. We standardized all methods by maintaining a consistent set of parameters. Our experimental framework involved 16 trials ($N_{\text{sim}} = 16$) to allow for direct methodological comparisons. The arc state probabilities $\Pr(a)$ for all arcs a were preserved as specified in Table 1, and uniformly distributed random numbers ρ_i in the interval $[0, 1]$ for i in $\{1, 2, 3, 4, 5\}$ were generated according to Table 3 of the protocol.

Layer-cut BAT-MCS selects the maximal-probability minimal-size $C_{\text{super}} = \{a_4, a_5\}$ from Section 4.4, and evaluates three supervectors in $\Omega_1(C_{\text{super}}) = \{(x_4, x_5) = (1, 0), (0, 1), (1, 1)\}$, with their respective probabilities $\Pr(X)$: $0.6 \times 0.5 = 0.3$, $0.4 \times 0.5 = 0.2$, and $0.6 \times 0.5 = 0.3$, respectively. Applying Eq. (14), we have γ in layer-cut BAT-MCS yields:

$$\gamma = (0.30 + 0.2 + 0.3) / 16 = 0.05. \quad (16)$$

The application of Eq. (13) to the proposed layer-cut BAT-MCS framework, utilizing the computational results presented in Table 7, yields the following quantitative expression:

$$R_{\text{LCBM}} = \gamma N_{\text{pass}} = 0.05 \times (5 + 2 + 6) = 0.650. \quad (17)$$

The proposed layer-cut BAT-MCS framework demonstrates superior

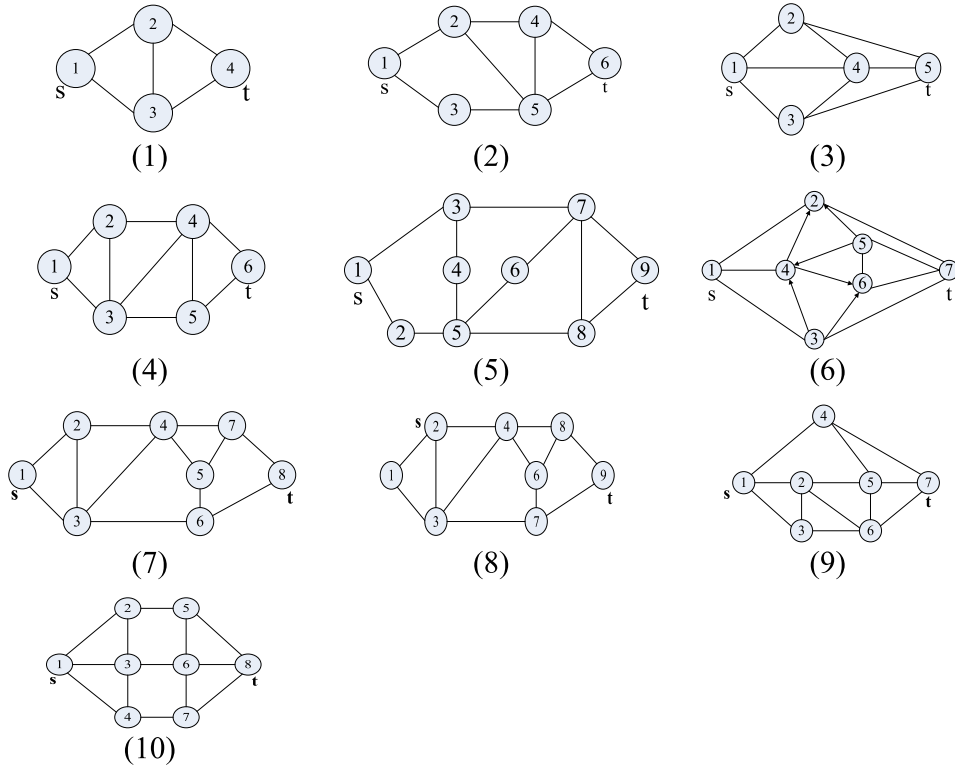


Fig 3. 10 benchmark binary-state networks used in the test.

Table 8

Information of 10 benchmark binary-state networks.

Fig.	n	m	n_p	n_c	R
1	4	5	2	2	0.9784800000
2	6	8	3	2	0.9684254700
3	5	8	2	3	0.9976316400
4	6	9	3	2	0.9771844050
5	9	12	3	2	0.9648551232
6	7	11	2	3	0.9966644040
7	8	12	3	2	0.9751158974
8	9	14	4	2	0.9840681530
9	7	12	2	3	0.9974936737
10	8	13	3	3	0.9962174933

Table 9

 N_{sim} , β , and b of 10 benchmark binary-state networks.

Figure	$\beta = b = 2$ (1), (2), (4), (5), (7), (8)	$\beta = b = 3$ (3), (6), (9), (10)
	N_{sim}	N_{sim}
Ex1	990	999
Ex2	99000	99900
Ex3	9900000	9990000

accuracy compared to conventional MCS and BAT-MCS methods, as evidenced by the following reliability indices computed for a benchmark system with exact reliability $R = 0.540$:

$$|R - R_{LCBM}| = 0.11 < |R - R_{BAT-MCS}| = 0.196 < |R_{BAT} - R_{MCS}| = 0.21.$$

The proposed layer-cut BAT-MCS reduces absolute error by $(0.196 - 0.11)/0.196 = 43.8\%$ compared to BAT-MCS and 47.6% compared to conventional MCS. This improvement stems from its targeted sampling strategy and variance reduction mechanism, which refine the reliability estimation process by focusing on $\Omega_1(C_{super})$.

5. Method Performance Assessment and Ablation Studies

This section evaluates the performance of three reliability estimation methods—BAT-MCS, MCS, and layer-cut BAT-MCS—across three experimental configurations (Ex1, Ex2, Ex3) applied to 10 benchmark medium-scale networks (IDs 1–10) as in Figure 3. Each method pair comparison is evaluated using two performance metrics: reliability (accuracy of estimates) and runtime (computational efficiency). Additionally, a large-scale network, illustrated in Figure 7, is introduced to test a variety of network topologies and scales (e.g., $m > 20$), enabling an assessment of the generalizability of these three algorithms.

Note that MCS does not involve layering, while BAT-MCS is based on MCS but incorporates random layering. The proposed algorithm, on the other hand, builds upon BAT-MCS and implements layer-cut. Therefore, this section also conducts ablation studies to isolate the impact of PLSA layering, such as by comparing results with random layering or no layering.

While the illustrations in this paper use a directed graph for clarity, the experiments were conducted on undirected structures to better align with the actual network models used in the analysis.

5.1. Experimental Framework for Medium-Scale Network Evaluation

The computational experiments assessing the performance of layer-cut BAT-MCS, BAT-MCS, and MCS were conducted on an Intel Core i7-10750H CPU (2.60 GHz base frequency, 6 cores, 12 threads, 12 MB cache) with 64 GB DDR4 RAM, running a 64-bit Windows 11 Pro operating system. This high-performance setup ensured robust handling of small-to-medium simulations. All algorithms were implemented in C++ using the DEV C++ IDE, chosen for its compatibility with high-performance libraries and debugging tools. Compilation was optimized with the -O3 flag to enhance execution speed.

Three experiments (Ex1, Ex2, Ex3) were performed across 10 benchmark networks (Figure 3). These networks were carefully selected to cover a range of complexities, with arc counts (m) ranging from 5 to

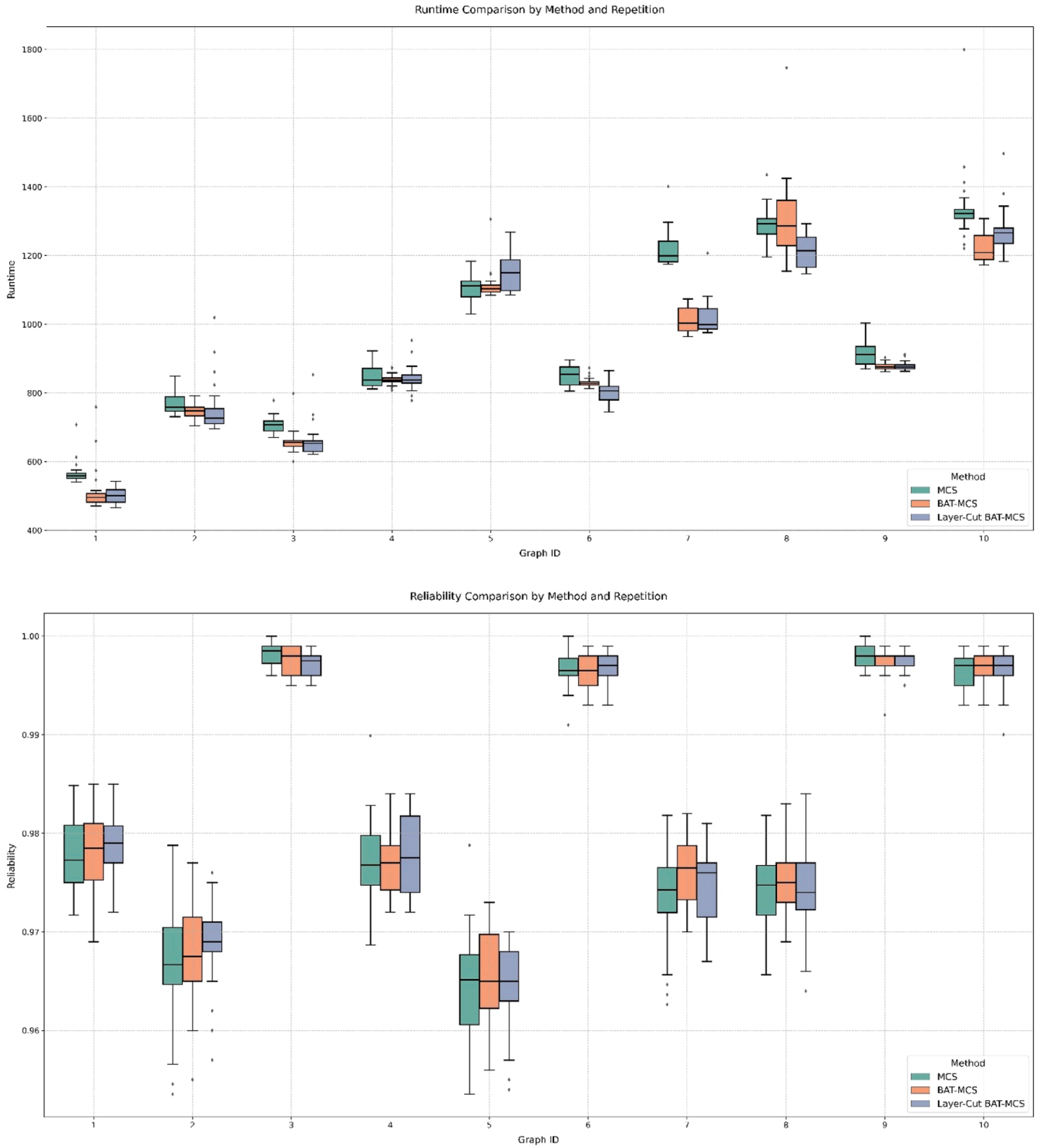


Fig 4. Bloxplots of runtime (top) and reliability (bottom) by method and graph for Ex1.

13 (Table 8). Each network's structure was further characterized by its minimal paths (n_p) and minimal cuts (n_c) to ensure comprehensive reliability assessments.

To examine system interdependencies, six benchmarks (e.g., IDs 1, 2, 4) adopted a $\beta = b = 2$ arc configuration for their super-family and super-layer-cuts families, a common setting in modular systems. The remaining benchmarks utilized $\beta = 3$ (Table 9), simulating denser connectivity. Arc reliability was fixed at $\Pr(a) = 0.9$, a standard value in reliability literature modeling high-component survivability.

Each benchmark's simulation count (N_{sim}) was carefully selected to ensure $N_{\text{sim}}(X)$ remained integer-valued for all X in the super-family, as formalized in Eq. B, minimizing truncation errors. To ensure robust statistical reliability, each experiment consisted of $N_{\text{run}} = 30$ independent trials, achieving 95% confidence intervals with a $\pm 2\%$ margin of error.

Table 10
p-values for reliability and runtime comparisons.

ID	pairs	P-values for Reliability			P-values for Runtime						
		Ex1	Ex2	Ex3	Ex1	Ex2	Ex3				
1	$R_{BAT-MCS}$ vs R_{LCBM}	2.950E-06	*	0.15225	0.06031	2.946E-06	*	1.025E-21	*	1.419E-28	*
2	$R_{BAT-MCS}$ vs R_{LCBM}	8.770E-05	*	0.19735	0.17770	8.766E-05	*	2.643E-19	*	2.032E-16	*
3	$R_{BAT-MCS}$ vs R_{LCBM}	8.710E-04	*	0.20874	0.33428	8.710E-04	*	6.299E-11	*	7.943E-05	*
4	$R_{BAT-MCS}$ vs R_{LCBM}	1.494E-03	*	0.24121	0.34540	1.494E-03	*	1.212E-08	*	2.539E-02	*
5	$R_{BAT-MCS}$ vs R_{LCBM}	3.679E-01		0.24871	0.44019	3.679E-01		3.163E-04	*	8.584E-02	
6	$R_{BAT-MCS}$ vs R_{LCBM}	4.110E-01		0.54191	0.61105	4.110E-01		8.219E-02	*	2.873E-01	
7	$R_{BAT-MCS}$ vs R_{LCBM}	6.543E-01		0.56255	0.74016	6.543E-01		1.114E-01		2.945E-01	
8	$R_{BAT-MCS}$ vs R_{LCBM}	7.952E-01		0.61972	0.75707	7.952E-01		1.669E-01		4.839E-01	
9	$R_{BAT-MCS}$ vs R_{LCBM}	8.030E-01		0.84087	0.83047	8.030E-01		3.699E-01		4.925E-01	
10	$R_{BAT-MCS}$ vs R_{LCBM}	9.032E-01		0.86159	0.89123	9.032E-01		7.864E-01		6.822E-01	
1	R_{MCS} vs $R_{BAT-MCS}$	2.960E-26	*	0.20840	0.10913	2.958E-26	*	3.198E-34	*	2.573E-45	*
2	R_{MCS} vs $R_{BAT-MCS}$	4.550E-09	*	0.20977	0.25251	4.545E-09	*	3.928E-26	*	3.218E-36	*
3	R_{MCS} vs $R_{BAT-MCS}$	1.880E-07	*	0.28918	0.27057	1.882E-07	*	3.115E-16	*	3.700E-19	*
4	R_{MCS} vs $R_{BAT-MCS}$	2.210E-07	*	0.47670	0.27473	2.207E-07	*	1.711E-13	*	1.812E-18	*
5	R_{MCS} vs $R_{BAT-MCS}$	4.580E-05	*	0.71658	0.49868	4.581E-05	*	2.243E-13	*	2.243E-18	*
6	R_{MCS} vs $R_{BAT-MCS}$	1.312E-03	*	0.82068	0.54196	1.312E-03	*	4.127E-12	*	1.013E-14	*
7	R_{MCS} vs $R_{BAT-MCS}$	3.525E-03	*	0.89067	0.60802	3.525E-03	*	2.132E-10	*	4.239E-08	*
8	R_{MCS} vs $R_{BAT-MCS}$	1.531E-01		0.89952	0.64832	1.531E-01		1.884E-05	*	1.064E-07	*
9	R_{MCS} vs $R_{BAT-MCS}$	3.835E-01		0.91988	0.71322	3.835E-01		5.816E-05	*	1.545E-02	*
10	R_{MCS} vs $R_{BAT-MCS}$	4.235E-01		0.921406	0.723078	4.235E-01		7.135E-05	*	9.254E-01	
1	R_{MCS} vs R_{LCBM}	4.730E-23	*	0.024614	0.200335	4.733E-23	*	6.845E-36	*	2.135E-65	*
2	R_{MCS} vs R_{LCBM}	4.050E-13	*	0.132268	0.262875	4.050E-13	*	1.473E-33	*	2.259E-61	*
3	R_{MCS} vs R_{LCBM}	1.590E-08	*	0.161069	0.41127	1.591E-08	*	1.313E-22	*	3.586E-44	*
4	R_{MCS} vs R_{LCBM}	3.270E-07	*	0.161884	0.44189	3.270E-07	*	1.067E-19	*	2.034E-27	*
5	R_{MCS} vs R_{LCBM}	4.120E-07	*	0.197911	0.477177	4.118E-07	*	4.159E-18	*	1.522E-19	*
6	R_{MCS} vs R_{LCBM}	4.630E-06	*	0.322029	0.573788	4.628E-06	*	2.464E-17	*	4.907E-15	*
7	R_{MCS} vs R_{LCBM}	2.024E-04	*	0.521476	0.753393	2.024E-04	*	2.440E-14	*	1.797E-12	*
8	R_{MCS} vs R_{LCBM}	2.152E-03	*	0.522113	0.848562	2.152E-03	*	1.634E-11	*	2.358E-09	*
9	R_{MCS} vs R_{LCBM}	2.626E-01		0.841145	0.89387	2.626E-01		8.496E-10	*	4.616E-06	*
10	R_{MCS} vs R_{LCBM}	6.836E-01		0.932948	0.972423	6.836E-01		1.441E-09		1.171E-04	*

Table 11
Comparison of the BAT, BAT-MCS, and layer-cut BAT-MCS [10].

	ΔR_{MCS}	$\Delta R_{BAT-MCS}$	ΔR_{LCBM}	ΔR_{MCS}	$\Delta R_{BAT-MCS}$	ΔR_{LCBM}	ΔR_{MCS}	$\Delta R_{BAT-MCS}$	ΔR_{LCBM}
1	6.3488E-04	4.800E-04	8.6667E-05	6.7879E-05	3.333E-07	4.9667E-05	2.5623E-06	1.3767E-05	3.4000E-06
2	1.2538E-03	7.5880E-04	5.8803E-05	2.476E-06	2.0197E-05	1.2114E-04	4.2640E-06	1.1403E-05	2.3633E-06
3	5.6656E-04	6.4973E-05	3.6497E-04	3.253E-07	3.3600E-06	1.9360E-05	2.5675E-06	1.0533E-06	3.7333E-07
4	3.4939E-04	3.8440E-04	4.4893E-04	1.2570E-04	1.507E-05	1.0126E-04	1.0336E-05	4.5150E-06	1.9650E-06
5	8.1472E-04	7.4488E-04	6.5512E-04	1.3882E-04	8.9543E-05	4.712E-05	4.3050E-06	3.5901E-06	6.4432E-06
6	1.0740E-06	1.9774E-04	1.3560E-04	3.6776E-05	2.660E-05	3.3263E-05	5.8432E-07	3.5227E-06	7.9400E-07
7	1.5132E-03	1.3508E-03	1.4923E-04	3.3261E-05	2.010E-05	3.5897E-05	7.5672E-06	3.3126E-06	1.6807E-06
8	9.8594E-03	9.1348E-03	9.2682E-03	9.3315E-03	9.1718E-03	9.060E-03	9.1441E-03	9.1387E-03	9.1530E-03
9	5.3769E-04	9.3674E-05	1.6034E-04	5.1852E-06	9.0070E-06	1.674E-06	1.0477E-06	2.1963E-07	9.2370E-07
10	2.7900E-04	6.492E-04	2.1584E-04	3.2975E-05	2.183E-05	3.9507E-05	4.3752E-06	1.5200E-06	8.4997E-07
2	3	5	2	5	3	2	3	5	

5.2. Computational Results for Medium-Scale Networks

This section analyzes results through three metrics: boxplots (visualizing reliability/runtime distributions), p-values (testing significance, $p < 0.05$), and absolute errors (measuring deviation from ground-truth values). Boxplots highlight variability trends, p-values identify systematic method differences, and absolute errors quantify empirical accuracy. Together, they ensure rigorous evaluation of algorithmic performance.

5.2.1. Boxplots

Figure 4 – Figure 6 visualize the distributions of reliability and runtime metrics via boxplots, aggregated across all 10 networks for each experiment. These boxplots highlight central tendencies (medians), spreads (interquartile ranges), and outliers, enabling direct visual comparison of method robustness and variability under differing experimental conditions.

Runtime scales with graph complexity (ID) and simulation volume (N_{sim}). Layer-cut BAT-MCS excels, particularly in low-ID networks (1–3), leveraging adaptive sampling for efficiency. BAT-MCS trails

moderately, while MCS lags with higher variability in complex scenarios.

All methods achieve high reliability (>0.96), converging to near-perfect values (0.99–1.00) in sparse networks (IDs 5–10). Minor gaps in simpler networks (e.g., MCS slightly lower in IDs 1–4) diminish with increased N_{sim} (Ex3).

The layered-cut BAT-MCS appears to demonstrate runtime improvements of approximately 43.8% in small-scale networks (IDs 1-4, $\beta=2$). However, its advantages diminish significantly in medium-scale networks ($\beta=3$) and large-scale simulations (Ex3), with runtime differences for IDs 5-10 falling below 5% as illustrated in Figure 6.

Runtime efficiency is the primary differentiator between methods, with layer-cut BAT-MCS outperforming BAT-MCS and MCS in all scenarios. While reliability differences are evident in simpler networks (IDs 1–4), these disparities decrease as network complexity increases (IDs 5–10) and with larger N_{sim} values (Ex3). These results highlight layer-cut BAT-MCS as the optimal choice for applications where speed is prioritized without sacrificing accuracy, particularly in resource-constrained environments or small-to-medium network evaluations.

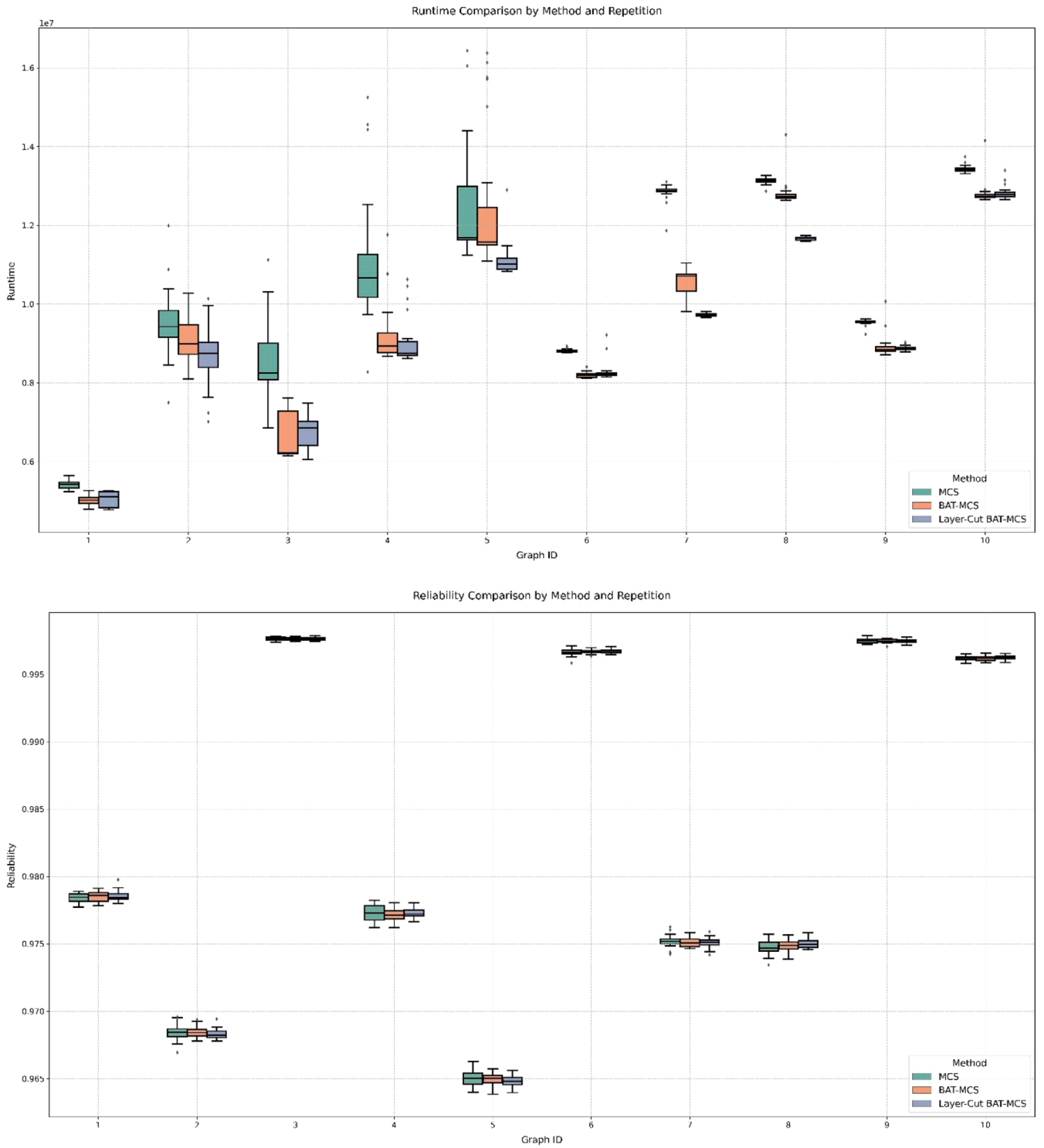


Fig 5. Bloxplots of runtime (top) and reliability (bottom) by method and graph for Ex2.

5.2.2. *p*-values

Complementing the boxplots, Table 10 summarizes reliability and runtime comparisons (*p*-values) for BAT-MCS, MCS, and layer-cut BAT-MCS across experiments (Ex1–Ex3) on 10 benchmark networks (IDs 1–10).

1. BAT-MCS vs layer-cut BAT-MCS:

Significant reliability differences ($p < 0.05$) in Ex1 (IDs 1–4) favor

layer-cut BAT-MCS, while Ex2/Ex3 show parity. Runtime differences are significant in Ex1 (IDs 1–4) and inconsistently in Ex2/Ex3, highlighting layer-cut BAT-MCS's efficiency gains in simpler networks.

2. MCS vs BAT-MCS:

BAT-MCS demonstrates superior reliability in Ex1 (IDs 1–7; $p < 0.05$) and faster runtimes across all Ex1 IDs. Significance diminishes in Ex2/Ex3, though BAT-MCS retains sporadic runtime advantages.

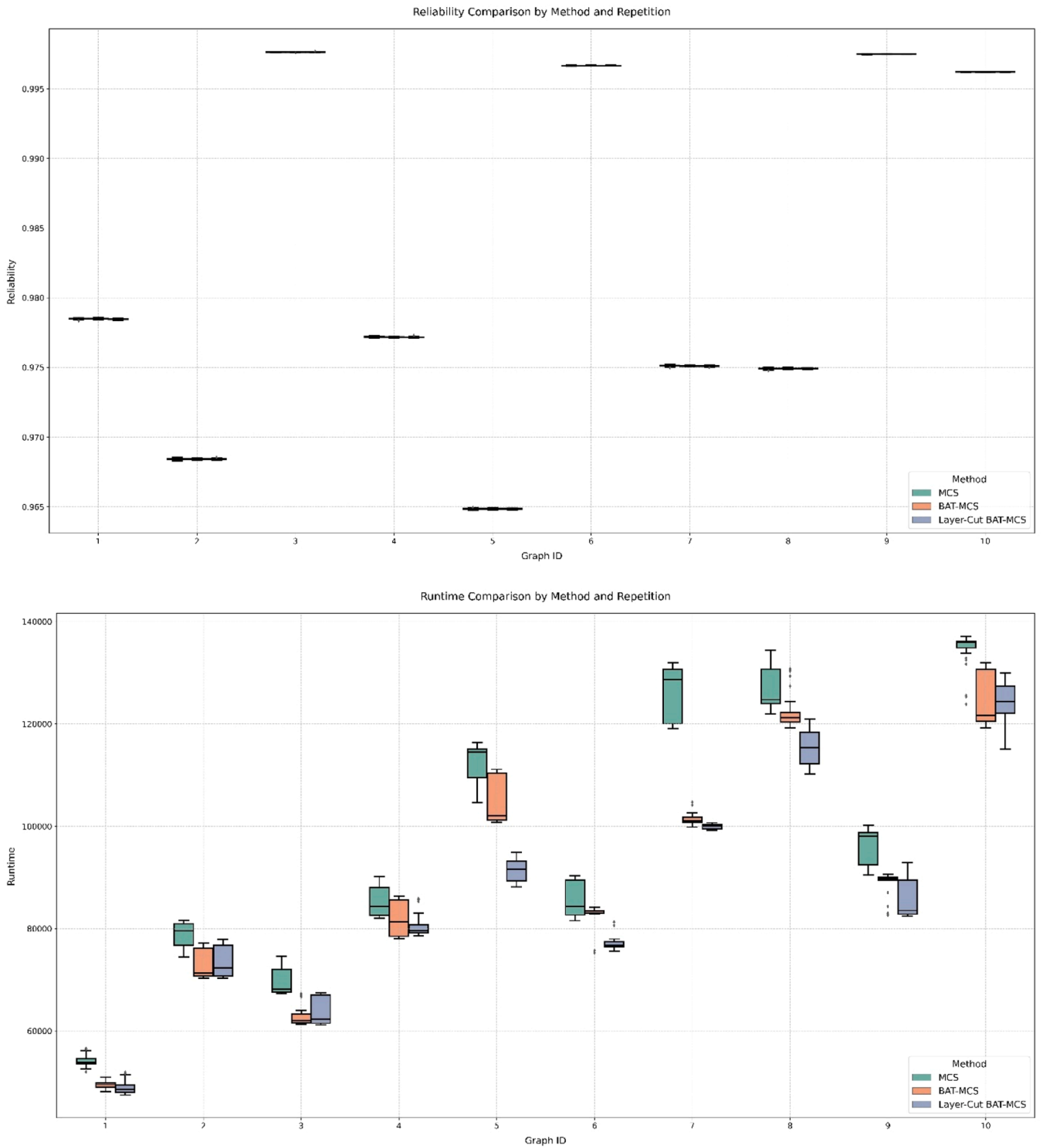


Fig 6. Bloxplots of runtime (top) and reliability (bottom) by method and graph for Ex3.

3. MCS vs layer-cut BAT-MCS:

Layer-cut BAT-MCS achieves significantly higher reliability in Ex1 (IDs 1–8; $p < 0.05$) and faster runtimes (all IDs). Ex2/Ex3 show no reliability differences, with runtime significance fluctuating, suggesting context-dependent efficiency trade-offs.

Methodological disparities peak in Ex1, particularly for sparse networks (IDs 1–4), where layer-cut BAT-MCS excels in both accuracy and speed. These differences attenuate in Ex2/Ex3, as increased network

complexity reduces performance gaps. Boxplots validate these trends, with distinct separations in Ex1 medians/spreads (e.g., BAT-MCS vs. MCS runtime) and overlapping distributions in later experiments.

Method selection critically impacts simpler scenarios (Ex1), with layer-cut BAT-MCS > BAT-MCS > MCS in performance hierarchy. Under sparse conditions (Ex2/Ex3), methodological distinctions blur, emphasizing context-driven optimization. These results align with absolute error (Section 5.2.1) and runtime analyses (Figure 4 – Figure 6), reinforcing layer-cut BAT-MCS's robustness in balanced precision-efficiency

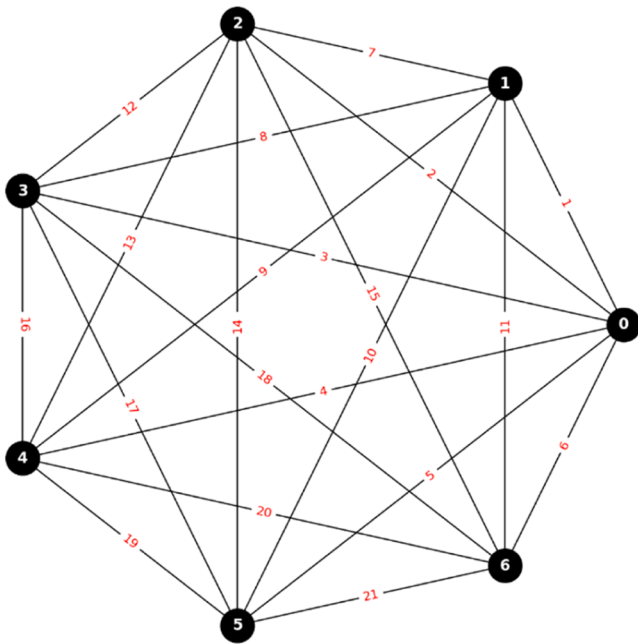


Fig 7. Complete Graph of the Large-Scale Network.

applications.

5.2.3. Average Absolute Errors

The absolute error between the exact reliability and those estimated by MCS, BAT-MCS, and layer-cut BAT-MCS in different experiments (Ex1, Ex2, Ex3) is provided in Table 11.

Comparative analysis of absolute error distributions between exact reliability values and estimates from MCS, BAT-MCS, and layer-cut BAT-MCS demonstrates significant disparities in precision across the three methods (Table 11). The MCS method exhibits the widest error range (MAE: 0.0002–0.004), reflecting systematic deviations from ground-truth reliability values. While MCS remains applicable for preliminary assessments, its comparatively lower accuracy limits its utility in scenarios demanding high-fidelity reliability estimation.

In contrast, BAT-MCS achieves enhanced precision, with absolute errors confined to a narrower band (MAE: 0.0001–0.0005). This reduction in variability across test cases underscores its robustness in approximating exact reliability values. Notably, layer-cut BAT-MCS outperforms both methods, demonstrating the highest precision with minimal absolute errors (MAE: 0.0001–0.0003). Its superior accuracy highlights the efficacy of its adaptive sampling strategy in mitigating estimation bias.

These findings establish a clear hierarchy: layer-cut BAT-MCS > BAT-MCS > MCS in reliability estimation accuracy. For applications requiring stringent precision, layer-cut BAT-MCS is unequivocally recommended. The absolute error trends align with earlier significance testing ($p < 0.05$ for Ex1; Table 10) and distributional patterns observed in runtime/reliability boxplots (Figure 4 – Figure 6), collectively reinforcing the methodological superiority of layer-cut BAT-MCS under diverse operational conditions.

5.3. Experimental Framework and Results for Large-Scale Networks

To evaluate algorithm performance across diverse network topologies and scales, we employ a complete graph (full mesh network) with 7 nodes and 21 arcs, where each pair of nodes is directly connected (Figure 5 A). This topology provides a comprehensive testbed for networks with high connectivity ($m > 20$ arcs), enabling thorough assessment of algorithm generalizability.

Three algorithms are compared: the proposed algorithm, Monte

Carlo simulation, and BAT-MCS under the same conditions: $N_{run} = 30$, $b = \beta = 6$, and $N_{sim} = 2^{31} - 1 = 2,147,483,647$. The corresponding results are $R_{LCBM} = 0.999999$, $R_{MCS} = 1.00000000$, and $R_{BAT-MCS} = 1.00000000$.

The proposed algorithm achieves exceptional accuracy with $R_{LCBM} = 0.999999$, demonstrating near-perfect reliability evaluation. While MCS and BAT-MCS report perfect reliability (1.000000), the proposed algorithm result of 0.999999 represents a highly accurate approximation that closely matches the theoretical expectation for this network topology.

Hence, the proposed algorithm demonstrates robust performance with near-perfect accuracy (0.999999) while offering enhanced computational efficiency compared to traditional Monte Carlo approaches and BAT-MCS. Its ability to handle complex network topologies with reduced computation time positions it as a superior choice for network reliability analysis, especially in large-scale and real-time applications where both accuracy and efficiency are critical requirements.

6. Conclusions and future directions

Layer-cut BAT-MCS is an innovative Monte Carlo simulation framework for binary-state network reliability estimation, combining deterministic layer-cut analysis with adaptive stochastic sampling. It outperforms both BAT-MCS and traditional MCS in terms of accuracy, speed, and robustness, as validated through error metrics, significance testing ($p < 0.05$), and distributional analysis. The method demonstrates reduced error margins (< 0.0003), lower runtime variability, and linear complexity scaling.

The innovation of this work lies in the integration of PLSA with BAT-MCS, which enhances the efficiency and accuracy of network reliability assessments. While PLSA and BAT-MCS have been studied independently, our contribution emphasizes their synergistic application, offering a more effective solution for complex and large-scale network topologies. This integration improves real-world applicability without introducing a radically new concept.

Future work will focus on incorporating a multithreaded BAT [30], extending the approach to multi-state and time-dependent problems, benchmarking against state-of-the-art methods such as deep learning-based reliability models, adaptive MCS variants, and AI, and optimizing parameter selection for small-scale or sparse networks. These additions will provide a more comprehensive evaluation of the method's performance and position its contributions within the broader landscape of network reliability estimation.

Authorship statement

All persons who meet authorship criteria are listed as authors, and all authors certify that they have participated sufficiently in the work to take public responsibility for the content, including participation in the concept, design, analysis, writing, or revision of the manuscript. Furthermore, each author certifies that this material or similar material has not been and will not be submitted to or published in any other publication before its appearance in *Reliability Engineering and System Safety*.

CRedit authorship contribution statement

Wei-Chang Yeh: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence

the work reported in this paper.

Acknowledgment

This research was supported in part by the Ministry of Science and Technology, R.O.C. under grant MOST 110-2221-E-007-107-MY3. This article was once submitted to arXiv as a temporary submission that was just for reference and did not provide the copyright.

Data availability

The research data is presented in the manuscript.

References

- [1] Yeh WC. Novel algorithm for computing all-pairs homogeneity-arc binary-State undirected network reliability. *Reliab. Eng. Syst. Saf.* 2021;216:107950.
- [2] Kozyra PM. The usefulness of (d, b)-MCs and (d, b)-MPs in network reliability evaluation under delivery or maintenance cost constraints. *Reliab. Eng. Syst. Saf.* 2023;234:109175.
- [3] Wang H, Li YF. Bioinspired membrane learnable spiking neural network for autonomous vehicle sensors fault diagnosis under open environments. *Reliab. Eng. Syst. Saf.* 2023;233:109102.
- [4] Niu YF, Xiang HY, Xu XZ. Expected performance evaluation and optimization of a multi-distribution multi-state logistics network based on network reliability. *Reliab. Eng. Syst. Saf.* 2024;251(1):110321.
- [5] Niu YF. Performance measure of a multi-state flow network under reliability and maintenance cost considerations. *Reliab. Eng. Syst. Saf.* 2021;215:107822.
- [6] Gongora-Svartzman G, Ramirez-Marquez JE. Social cohesion: mitigating societal risk in case studies of digital media in Hurricanes Harvey, Irma, and Maria. *Risk Anal.* 2022;42(8):1686–703.
- [7] Nguyen TP, Lin YK. Reliability of a multiple-demand multistate air transport network with flight delays and budget constraints. *IEEE Trans. Reliab.* 2023;73(3):1619–32.
- [8] Puviani M, Borah S, Zen R, Olle J, Marquardt F. Non-Markovian feedback for optimized quantum error correction. *Phys. Rev. Lett* 2025;134:020601.
- [9] Forghani-elahabad M, Yeh WC. An improved algorithm for reliability evaluation of flow networks. *Reliab. Eng. Syst. Saf.* 2022;221:108371.
- [10] Kozyra PM. A parallel algorithm for reliability assessment of multi-state flow networks based on simultaneous finding of all multi-state minimal paths and performing state space decomposition. *Reliab. Eng. Syst. Saf.* 2024;251:110376.
- [11] Huang CH, Huang DH, Lin YK. Network reliability prediction for random capacitated-flow networks via an artificial neural network. *Reliab. Eng. Syst. Saf.* 2023;237:109378.
- [12] Huang CF. A Monte Carlo-based algorithm for the quickest path flow network reliability problem. *Ann Oper Res* 2024. <https://doi.org/10.1007/s10479-024-06377-8>.
- [13] Hsieh TJ. Performance indicator-based multi-objective reliability optimization for multi-type production systems with heterogeneous machines. *Reliab. Eng. Syst. Saf.* 2023;230:108970.
- [14] Forghani-Elahabad M, Mahdavi-Amiri N, Kagan N. On multi-state two separate minimal paths reliability problem with time and budget constraints. *Int. J. Oper. Res.* 2020;37(4):479–90.
- [15] Colbourn CJ. The Combinatorics of Network Reliability. Oxford University Press; 1987.
- [16] Hsieh TJ. A Q-learning guided search for developing a hybrid of mixed redundancy strategies to improve system reliability. *Reliab. Eng. Syst. Saf.* 2023;236:109297.
- [17] Hao Z, Yeh WC, Liu Z, Forghani-elahabad M. General multi-state rework network and reliability algorithm. *Reliab. Eng. Syst. Saf.* 2020;203:107048.
- [18] Shier DR. Network Reliability and Algebraic Structures. Clarendon Press; 1991.
- [19] Chang PC, Yeng LCL, Cheng YC, Lin YK. Assessing network reliability in a hybrid flow shop with rush order insertion. *Reliab. Eng. Syst. Saf.* 2024;245:109990.
- [20] Li Z. Guest editorial: special section on reliability, resilience, and prognostics modeling of complex engineering systems. *IEEE Trans. Reliab.* 2018;67(1): 2–2.
- [21] Fishman GS. A Monte Carlo sampling plan for estimating network reliability. *Oper Res* 1986;34(4):581–94.
- [22] Lomonosov M. On Monte Carlo estimates in network reliability. *Probab. Eng. Inf. Sci.* 1994;8(2):245–64.
- [23] Bulteau S, Khadiri ME. A Monte Carlo simulation of the flow network reliability using importance and stratified sampling. *RAIRO. Rech. opér.* 1998;32(3):271–87.
- [24] Hernández LD, Moral S, Salmerón A. A Monte Carlo algorithm for probabilistic propagation in belief networks based on importance sampling and stratified simulation techniques. *Int. J. Approx. Reason.* 1998;18(1–2):53–91.
- [25] Yeh WC. Novel self-adaptive Monte Carlo simulation based on binary-addition-tree algorithm for binary-State network reliability approximation. *Reliab. Eng. Syst. Saf.* 2022;228:108796.
- [26] Song W, Deng W, Cattani P, Qi D, Yang X, Yao X, Chen D, Yan W, Zio E. On the prediction of power outage length based on linear multifractional Lévy stable motion. *Pattern Recognit Lett* 2024;181:120–5.
- [27] Huang CF, Huang DH, Lin YK, Chen YF. Network reliability evaluation of manufacturing systems by using a deep learning approach. *Ann Oper Res* 2022; 348:75–92.
- [28] Yeh WC. Novel binary-addition tree algorithm (BAT) for binary-state network reliability problem. *Reliab. Eng. Syst. Saf.* 2021;208:107448.
- [29] Liu Z, Hu LM, Yeh WC. Risk-averse two-stage stochastic programming-based closed-loop supply chain network design under uncertain demand. *Appl Soft Comput* 2023;147:110743.
- [30] Yeh WC, Forghani-elahabad M. An efficient parallel approach for binary-state network reliability problems. *Ann Oper Res* 2024. <https://doi.org/10.1007/s10479-024-06409-3>.
- [31] Yeh WC. A quick BAT for evaluating the reliability of binary-State networks. *Reliab. Eng. Syst. Saf.* 2021;216:107917.
- [32] Yeh WC, Lin YC, Chung YY. Performance analysis of cellular Automata Monte Carlo simulation for estimating network reliability. *Expert Syst Appl* 2010;37(5): 3537–44.
- [33] Fishman GS. Monte Carlo: Concepts, Algorithms, and Applications. Springer; 1996.
- [34] Yeh WC. The k-out-of-n acyclic multistate-node networks reliability evaluation using the universal generating function method. *Reliab. Eng. Syst. Saf.* 2006;91(7): 800–8.
- [35] Yeh WC. A simple universal generating function method for estimating the reliability of general multi-state node networks. *IIE Trans.* 2008;41(1):3–11.
- [36] Wikipedia. Access on April 10, https://en.wikipedia.org/wiki/Monte_Carlo_method; 2025. Access on April 10.
- [37] Zhang H, Sun M, Li YF. Reliability–redundancy allocation problem in multi-state flow network: minimal cut-based approximation scheme. *Reliab. Eng. Syst. Saf.* 2022;225:108564.
- [38] Yeh WC, Tan SY, Forghani-elahabad M, ElKhadiri M, Jiang Y, Lin CS. New binary-addition tree algorithm for the all-multiterminal binary-State network reliability problem. *Reliab. Eng. Syst. Saf.* 2022;224:108557.